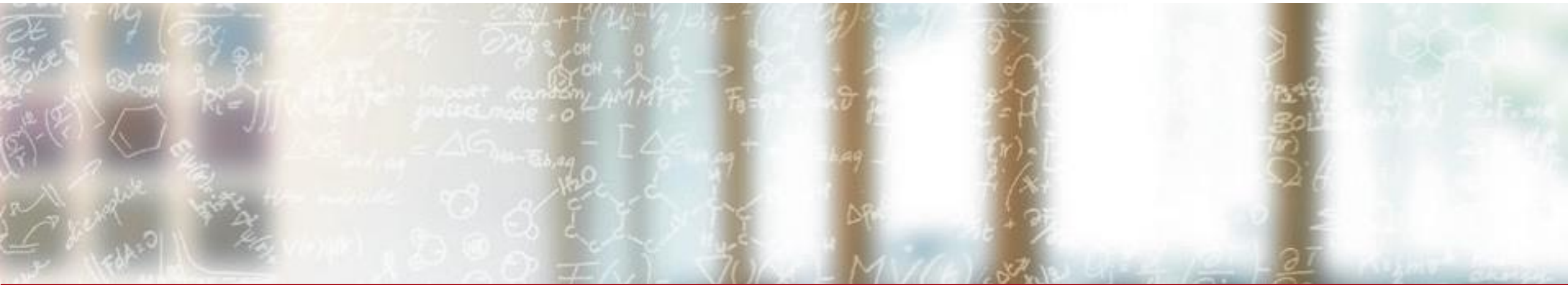




CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Inference Service

CSCS User Lab Day 2026

David Isztl, CSCS

August 28, 2026

Table of Contents

1. What is an inference service

- The mental model: an HTTP endpoint, and what makes it different from training

2. Our infrastructure

- K8s, KServe, Envoy, Envoy AI Gateway, KEDA, ILM-d + EPP, and the GH200 hardware underneath

3. Model hardware requirements

- Single GPU, tensor parallel, tensor + pipeline parallel, and how to size GPU memory

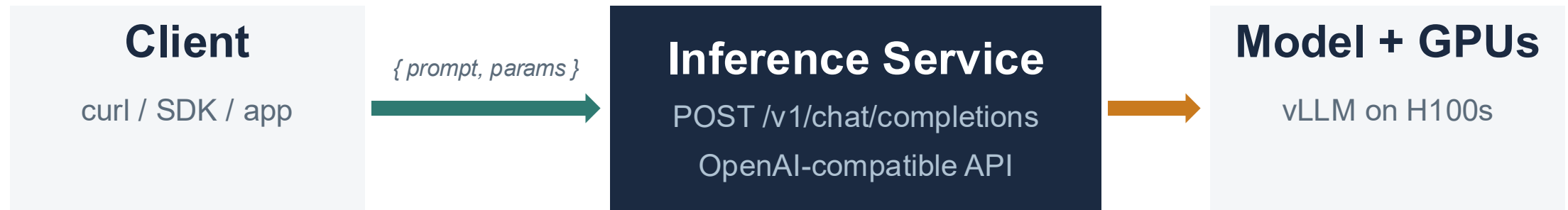
4. Models we offer & pricing

- The catalog, how to pick a model, and what it costs

What is an inference service

It's an HTTP(S) endpoint

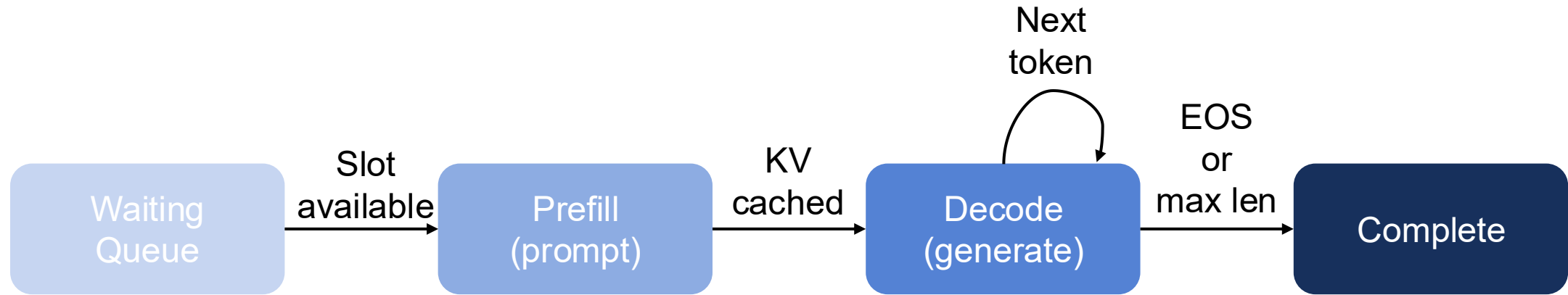
"https://api.inference.cscs.ch/v1"



One **consistent request/response contract**, regardless of which of our 10 models answers it

Requests are stateless per call; the model itself holds **no memory** of prior turns unless the client resends history

Request lifecycle



Prefill is **compute-bound**: throughput scales with FLOPs, done once per request.

Decode is **memory-bandwidth-bound**: every step re-reads the full KV cache from HBM.

Serving is not training

	Training	Inference
Workload shape	Offline, scheduled batch jobs	Online, arrives whenever users show up
Optimize for	Total throughput (tokens/GPU/day)	Latency per request + cost per token
Duration	Days to weeks per run	Milliseconds to a few seconds per request
Concurrency	One large , fixed batch	Many small , bursty, unpredictable requests
Failure cost	Restart from checkpoint	User sees it immediately: availability matters

The metrics that define "good"

TTFT

Time To First Token:
latency from request to
the first streamed token
(dominated by prefill)

TPOT / ITL

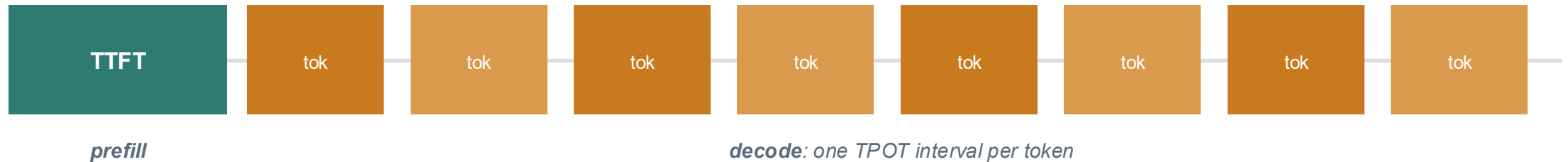
Time Per Output Token:
inter-token latency during
decode; sets perceived
"typing speed"

Throughput

Tokens/sec and
requests/sec the
deployment sustains
under concurrent load

Availability

Uptime and **error rate** of
the endpoint: what SLOs
are measured against



What we actually watch on the dashboards.



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

Our infrastructure

CSCS - Swiss National Supercomputing Centre



- Lugano, Switzerland
- Founded in 1991
- “We develop and operate a **High Performance Computing** and **data research infrastructure** that supports **world-class science** in Switzerland.”
- ~ 140 employees
 - ~ 100 in Lugano
 - ~ 40 in Zurich
 - ~ 22 nationalities

ALPS

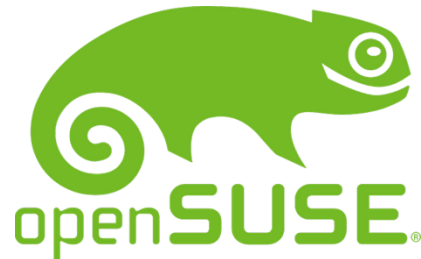


OpenSource Software



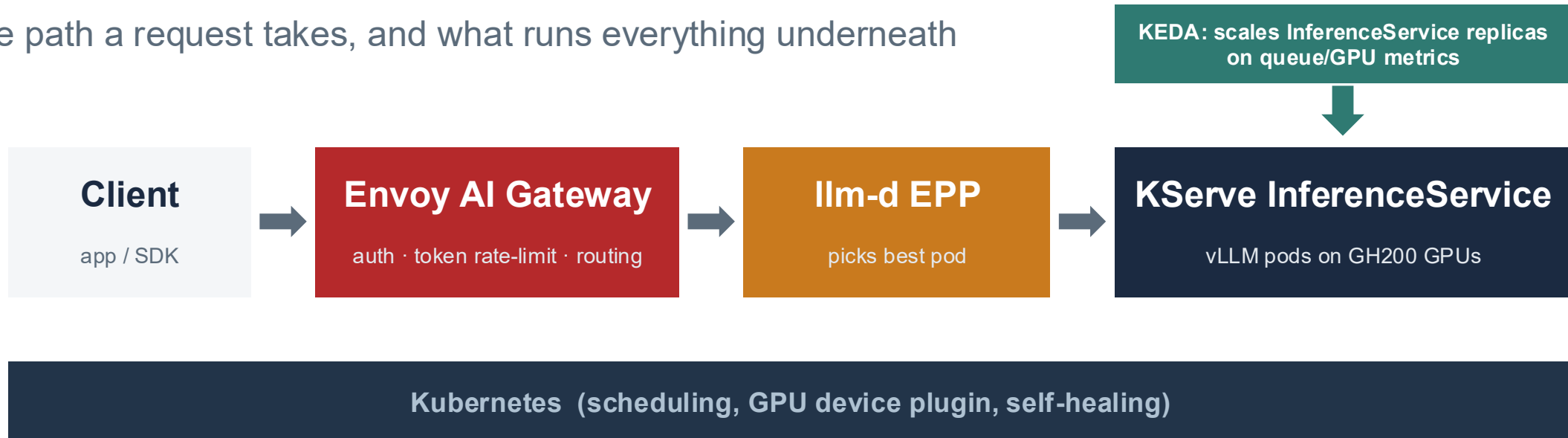
NVIDIA/nccl

Optimized primitives for collective multi-GPU communication



Architecture at a glance

The path a request takes, and what runs everything underneath



External traffic always enters through **Envoy AI Gateway**

llm-d's EPP chooses the specific replica per request (**not round-robin**)

Everything is a Kubernetes workload; KEDA and KServe manage the GPU pod lifecycle

Kubernetes

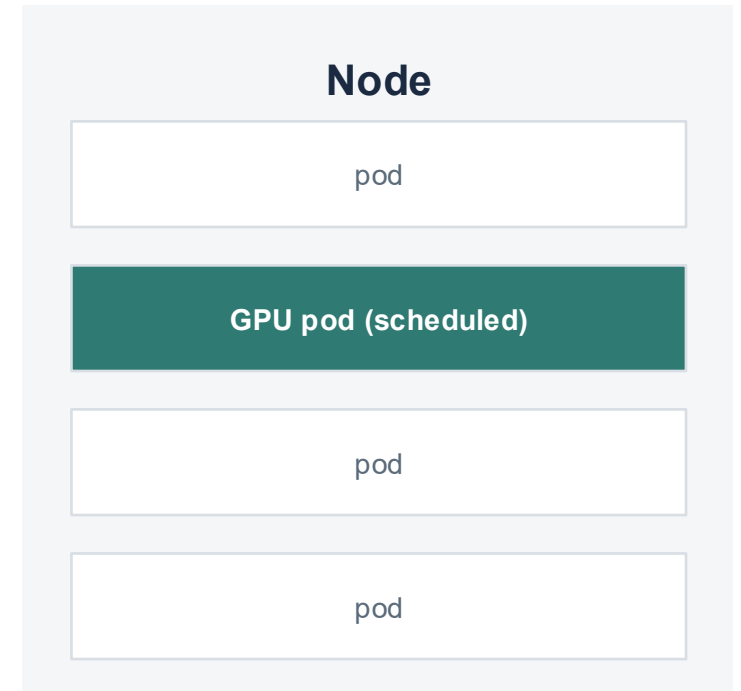
One orchestration layer for the entire inference service.

GPU scheduling: the NVIDIA device plugin exposes GPUs as a schedulable resource; pods request exactly the GPUs they need.

Declarative state: we describe the desired deployment; Kubernetes continuously reconciles reality to match it.

Self-healing: a crashed pod or a drained node is automatically rescheduled, without manual intervention.

Common substrate: the same cluster primitives underpin every layer discussed next: KServe, Envoy, KEDA, ILM-d.



Kserve: model serving as a code

InferenceService CRD: declares model, serving runtime (vLLM), resources, and autoscaling in one object.

Decouples model: GPU pods not directly managed.

Runtime-agnostic: standardizes serving across different backends, so onboarding a new model is templated.

Rollout support: rollout of a new model version without downtime.

```
217   apertus-8b-instruct:
218     uri: "hf://swiss-ai/Apertus-8B-Instruct-2509"
219     release: apertus-8b
220     storage: 20Gi
221     replicas: 1
222     tensorParallelSize: 1
223     maxModelLen: 32768
224     maxNumSeqs: 64
225     dshm: 16Gi
226     extraArgs:
227       - --trust-remote-code
228     resources:
229       requests:
230         cpu: "8"
231         memory: 32Gi
232       limits:
233         cpu: "16"
234         memory: 64Gi
235       gpu: 1
236     gpuMemoryUtilization: "0.90"
237     startupProbe:
238       initialDelaySeconds: 300
239       failureThreshold: 30
```


Envoy

L7 proxy: understands HTTP/gRPC, not just TCP, can route on path, headers, method.

TLS termination: handles encryption at the edge so backend pods don't have to.

Load balancing: distributes connections across healthy backend pods.

Observability: emits metrics, logs and traces for every request that passes through it.

Envoy

the plain L7 proxy

Envoy AI Gateway (next slide)
adds LLM-specific smarts on top

Envoy AI Gateway

LLM-aware routing on top of Envoy

Token-based rate limiting: limits are set in tokens/min, not just requests/sec

Unified API surface: one OpenAI-compatible endpoint in front of 10 different models/backends.

Auth & API keys: authenticates callers and maps them to quota and billing before any GPU is touched.

Model-aware routing: reads the request body to route by requested model name to the right backend.

Envoy AI Gateway

auth

token-aware rate limits

routes to llm-d EPP

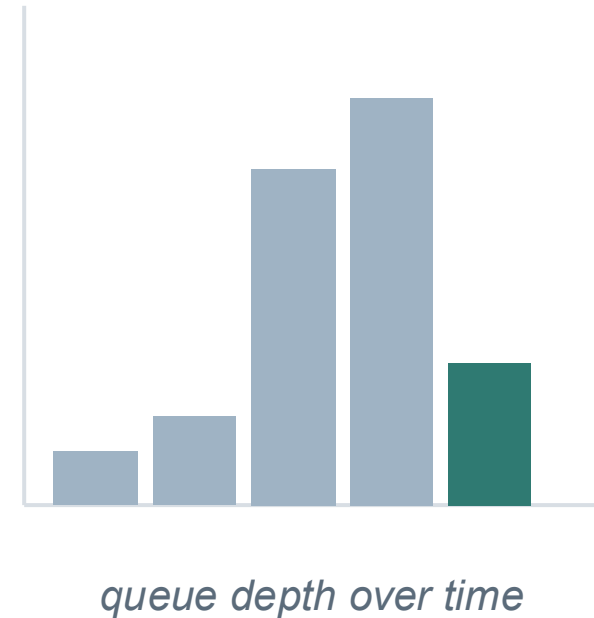
KEDA: event-driven autoscaling

Beyond CPU: scales on queue depth, in-flight requests, or GPU/KV-cache utilization

Scales InferenceServices directly: adds/removes GPU replicas of the exact model under load.

Scale-to-zero: idle models release their GPUs entirely, freeing hardware for the fleet.

Fast reaction: polls custom metrics frequently enough to catch bursty LLM traffic patterns.



Ilm-d + EPP: smart request placement

EPP (Endpoint Picker Provider): sits after the gateway and picks the specific pod for each request.

Cache-aware: prefers a pod that already has the request's prefix in its KV cache, cutting prefill cost.

Load-aware: also factors in current queue/utilization per pod, smarter than round-robin.

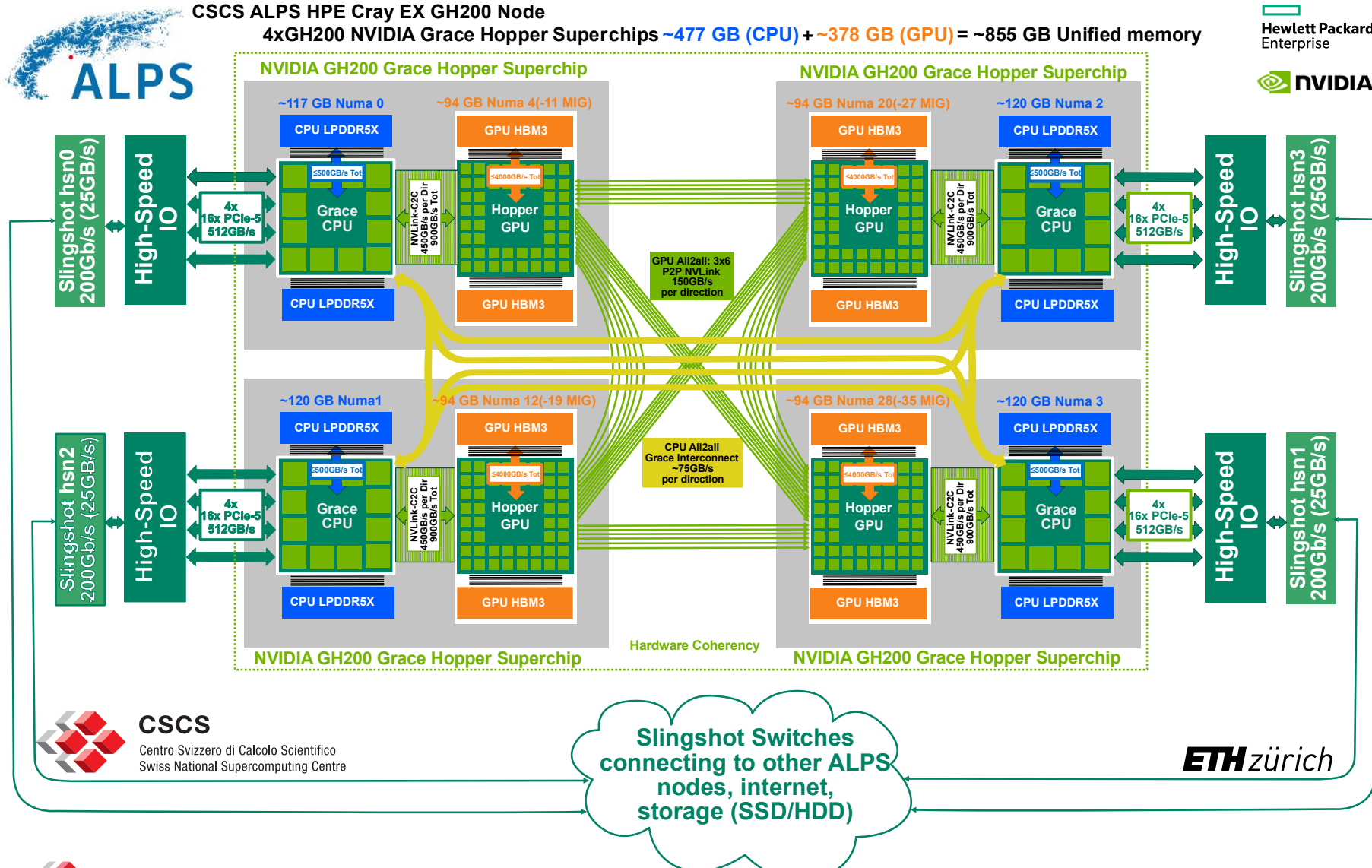
EPP decision

Pod A — cache hit, low load

Pod B — cold cache, busy

Pod C — cold cache, idle

Anatomy of a GH200 node

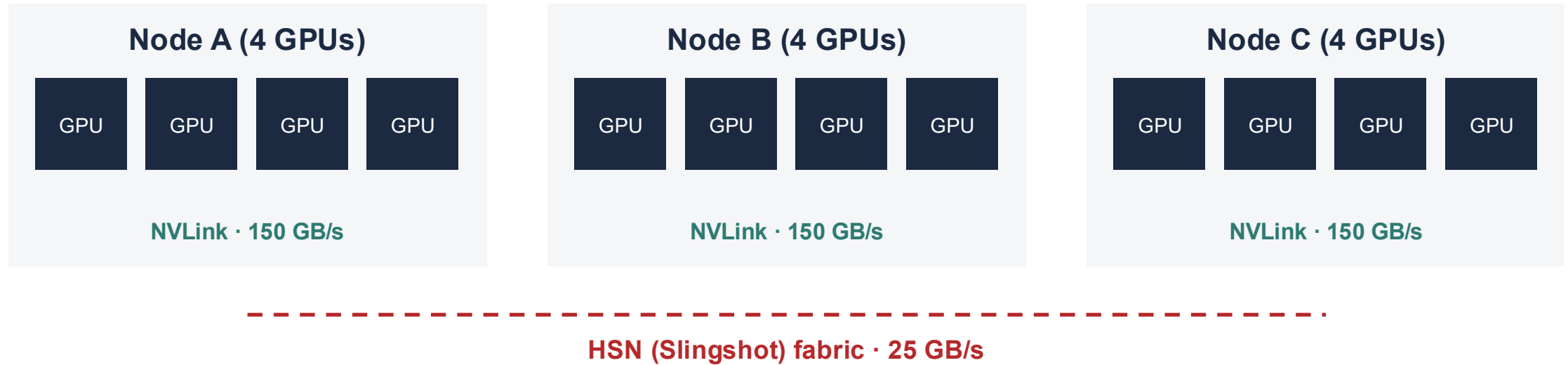


2688 GH200 Nodes
 4 GPUs per node
 ~94GB per GPU

Intra-node speed:
 NVLink 150GB/s

Inter-node speed:
 HSN 25GB/s

Anatomy of a GH200 node



NVLink intra-node (150 GB/s) is ~6x faster than the HSN inter-node fabric (25 GB/s)

tensor parallelism is confined to a node, and pipeline parallelism spans nodes



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

Model hardware requirements

How weights take up space

$$\text{weight size (GB)} = \text{parameter count} \times \text{bytes per parameter}$$

FP32
4 bytes

Full precision.

BF16 / FP16
2 bytes

Common serving default.

FP8
1 byte

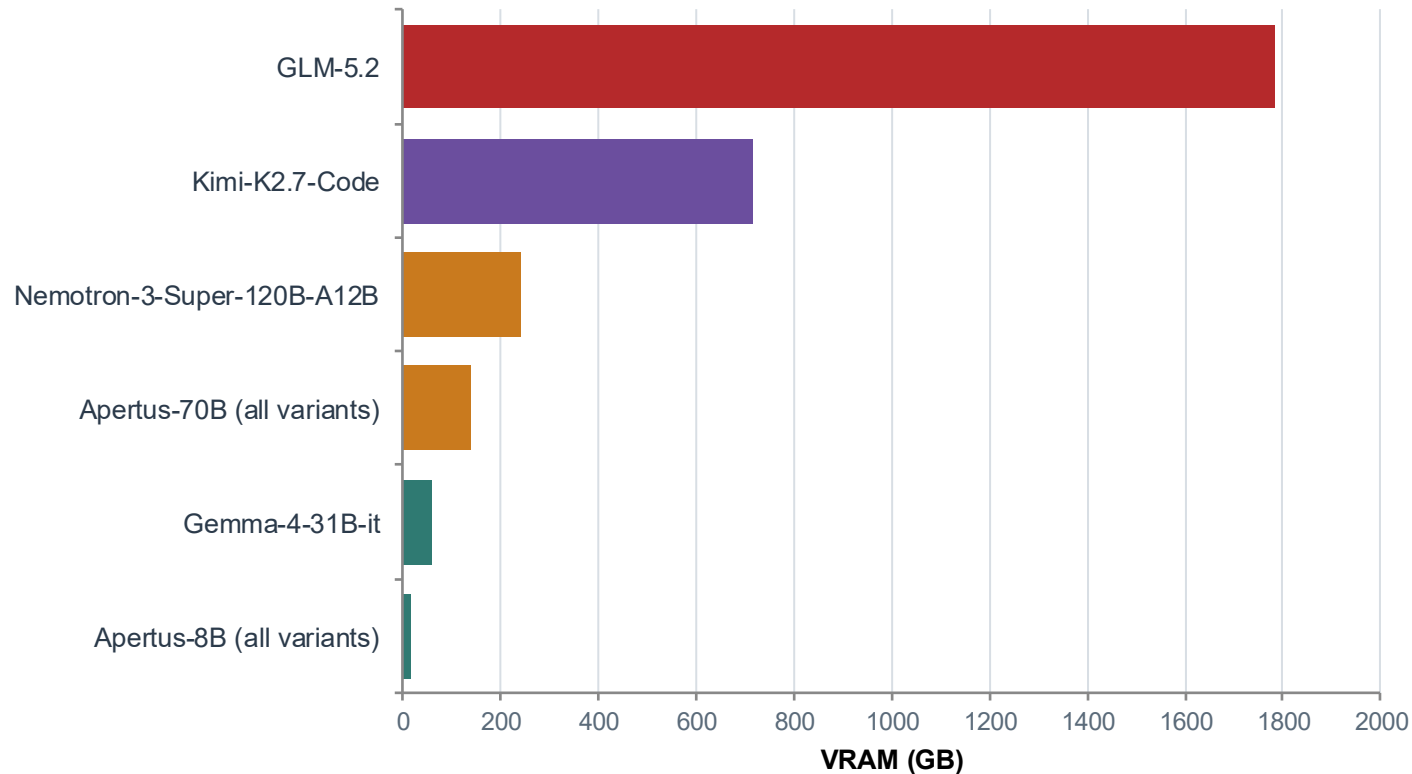
Half the footprint of
BF16, small quality
trade-off.

Int4
0.5 bytes

Smallest footprint.
Requires quantization-
aware serving.

Checkpoints mix precisions: most layers may run at a low precision while embeddings, output heads, or normalization layers stay higher for quality.

Memory footprint across our served fleet



Parameter count vs. load size

Kimi-K2.7-Code: 1T params, 714 GB to load:
about 0.71 effective bytes/parameter.

GLM-5.2: 743B params, 1'786 GB to load:
about 2.4 effective bytes/parameter, closer to
full BF16.

**So GLM-5.2, with fewer parameters, still
needs the larger footprint**

KV cache is not included

Single GPU, tensor parallel, pipeline parallel

Single GPU

Entire model on one H100.
Simplest. No cross-GPU communication at all.

Weights + KV fit in 96 GB

Tensor Parallel (TP)

Each layer's weights are split across GPUs; every forward pass needs a fast all-reduce between them.

Needs NVLink: stays inside one node

TP + Pipeline Parallel (PP)

TP inside each node; different layers/stages live on different nodes, connected by PP.

PP tolerates the slower HSN fabric across nodes

TP scales out within the 150 GB/s NVLink domain

PP is how you cross into the 25 GB/s HSN domain without killing throughput.

Anatomy of GPU memory

weights + KV cache $\leq$ 90% x total GPU memory

activations & framework overhead = 10% x total GPU

Weights

total_params x effective
bytes/param, at the model's actual
load size

KV cache

Fills whatever remains of the 90%
envelope after weights. This is the
pool vLLM's PagedAttention
allocates blocks from.

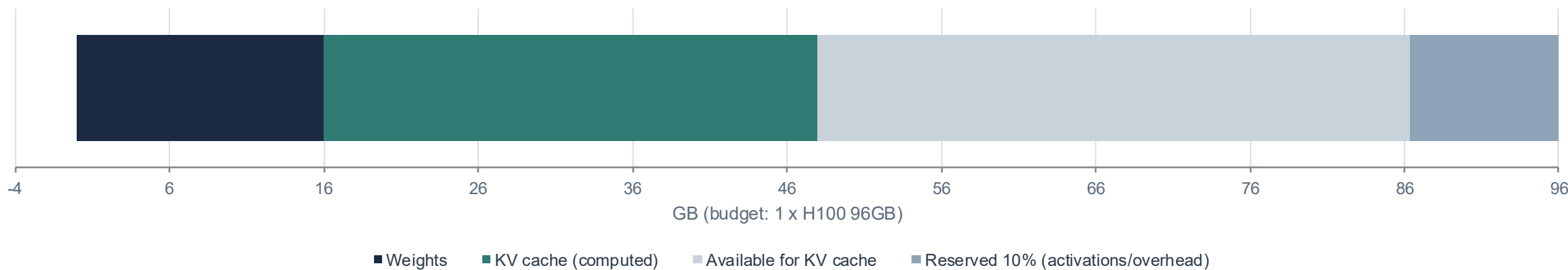
Reserved 10%

vLLM's gpu_memory_utilization
defaults to 0.9

An example of model weights allocation: Apertus-8B-Instruct-2509

32 layers · hidden 4096 · GQA 8 KV heads · head_dim 128 · BF16 weights + KV · batch 8 @ 32'768 tok

This model never leaves the single-GPU tier, even at production concurrency.

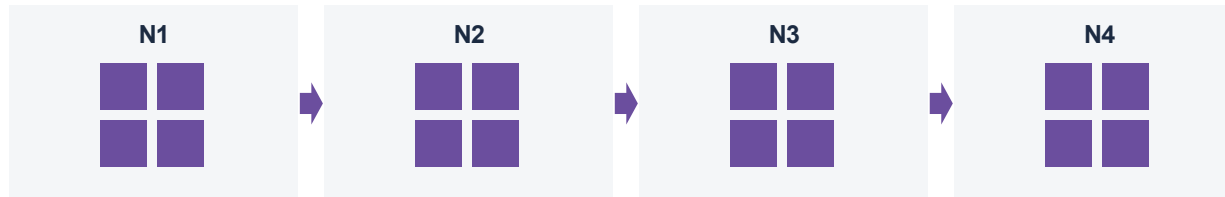


Weights 16 GB + KV 32 GB used -> 38 GB more KV-cache capacity within the 90% envelope | 10 GB reserved (fixed 10%)

Tensor + Pipeline Parallel: GLM-5.2 and Kimi-K2.7-Code

Our production topology: TP=4 per node, PP across nodes

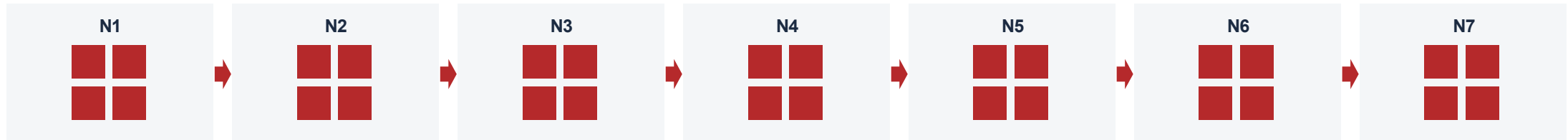
Kimi-K2.7-Code: 1T params



714 GB to load -> TP=4 x PP=4 = 16 GPUs, 4 nodes

1,536 GB total capacity · 822 GB left for KV cache + reserve

GLM-5.2: 743B params



1,786 GB to load -> TP=4 x PP=7 = 28 GPUs, 7 nodes

2,688 GB total capacity · 902 GB left for KV cache + reserve

Node count tracks load size, not parameter count: GLM-5.2's higher effective bytes/parameter gives it a larger footprint than Kimi-K2.7-Code, so it needs proportionally more nodes to serve, even with fewer total parameters.

Models we offer & pricing

The full catalog

Model	Context	Tier
Apertus-8B-Instruct-2509	32'768	Single GPU
Apertus-v1.5-8B	262'144	Single GPU
Apertus-v1.5-8B-thinking	262'144	Single GPU
Gemma-4-31B-it	262'144	Single GPU
Apertus-70B-Instruct-2509	64'000	TP (intra-node)
Apertus-v1.5-70B	262'144	TP (intra-node)
Apertus-v1.5-70B-thinking	262'144	TP (intra-node)
Nemotron-3-Super-120B-A12B	262'144	TP (intra-node)
GLM-5.2	976'000	TP+PP (inter-node)
Kimi-K2.7-Code	262'144	TP+PP (inter-node)

Pricing

Model	CHF / 1M input tok	CHF / 1M output tok
swiss-ai/Apertus-70B-Instruct-2509	0.06	0.40
swiss-ai/Apertus-8B-Instruct-2509	0.01	0.04
swiss-ai/Apertus-v1.5-70B	0.06	0.40
swiss-ai/Apertus-v1.5-8B	0.01	0.04
moonshotai/Kimi-K2.7-Code	1.30	6.05
zai-org/GLM-5.2 [*]	6.36	27.31
google/gemma-4-31B-it	0.06	0.42
nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16	0.30	1.49

^{*}replaced soon by GLM-5.3 with improved capabilities and lower price per token

How to pick a model

- | | |
|---|--|
| ● Small, latency-sensitive task? | Apertus-8B family: cheapest, single GPU, lowest latency |
| ● Balanced quality/cost, moderate context? | Apertus-70B or gemma-4-31B-it |
| ● Frontier reasoning, agentic, or very long context? | GLM-5.2 or Kimi-K2.7-Code: highest capability, highest footprint |
| ● Multilingual / Swiss-compliance requirement? | Apertus family (trained on fully open, compliant data) |

Getting started

ui.inference.cscs.ch



CSCS Inference API

 Sign In with CSCS Account

Use your CSCS Keycloak account to sign in.

Sign in with access token

Find your key group



STAFF

Key Groups

Pricing

Key Groups

Updated Aug 27, 10:05

SWISS NATIONAL SUPERCOMPUTING CENTRE

>

cscs-ws-lram-inference-6

Portal

ws-lram Swiss National Supercomputing Centre 0f988a99... Monthly

3 keys

>

cscs-ws-lam-1-inference-5

Portal

ws-lam Swiss National Supercomputing Centre be01af64... Monthly

1 key

>

cscs-chatterbox-inference-4

Portal

Chatterbox Swiss National Supercomputing Centre 6122f8a2... Monthly

0 keys

>

cscs-csstaff-inference-5-s

Portal

Paused

csstaff Swiss National Supercomputing Centre d6ee2c99...

2 keys

>

cscs-csstaff-inference-4-2-RI-eng-unit

Portal

csstaff Swiss National Supercomputing Centre c0bed3d7... Monthly

1 key

>

cscs-csstaff-inference-war-room-t1

Portal

csstaff Swiss National Supercomputing Centre 8eaa7b55... Monthly

3 keys

SWISSAI INITIATIVE

>

cscs-apertus-team-inference

Portal

intra01 SwissAI Initiative 0112e983... Monthly

3 keys

>

swissai-prep23-inference-4

Portal

prep23 SwissAI Initiative e4cf6266... Monthly

1 key

UNIVERSITY OF ZURICH

>

uzh-uzh0-inference-5

Portal

uzh0 University of Zurich b08e04e1... Monthly

1 key

Getting started

Create a key

Create Key for isztld

KEY ALIAS

my-key

MODELS

☐ ALL

☐ google/gemma-4-31B-it

☒ moonshotai/Kimi-K2.7-Code

☐ nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16

☐ swiss-ai/Apertus-70B-Instruct-2509

+ Create Key

Cancel

Use the desired model

```
(base) [isztld @ Davids-MacBook-Pro]-[11:41:32.014]-[/Users/isztld]:-)
curl -s -X POST https://api.inference.cscs.ch/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer [REDACTED]" \
  -d '{
    "model": "moonshotai/Kimi-K2.7-Code",
    "messages": [
      {"role": "user", "content": "Hello"}
    ]
  }'
```

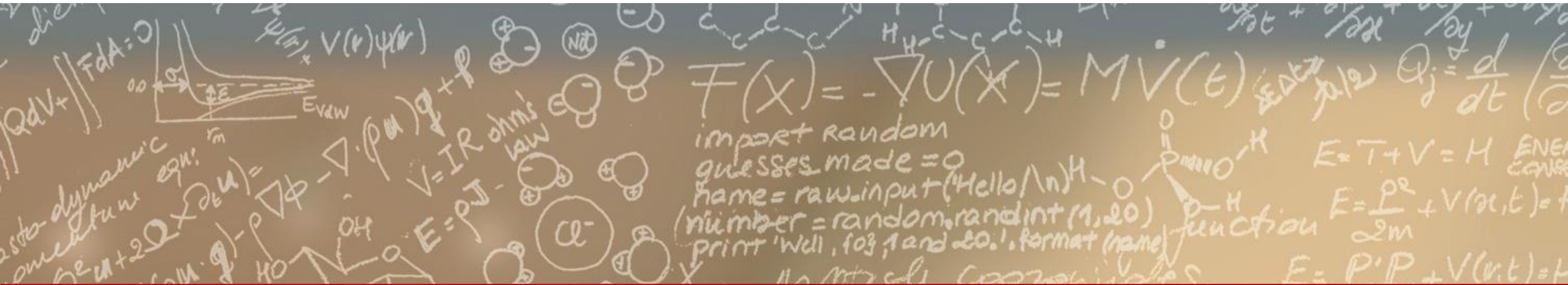
```
[{"id": "chatcpl-5530788b-f32b-4b9c-b3bf-a681e139c8b7", "object": "chat.completion", "created": 1787650895, "model": "moonshotai/Kimi-K2.7-Code", "choices": [{"index": 0, "message": {"role": "assistant", "content": "Hello! How can I help you today?", "refusal": null, "annotations": null, "audio": null, "function_call": null, "reasoning": "We need answer to greeting 'Hello'. Should be friendly, helpful, ask how can assist. Keep it simple."}, "logprobs": null, "finish_reason": "stop", "stop_reason": 163586, "token_ids": null, "routed_experts": null}], "service_tier": null, "system_fingerprint": "vllm-0.27.2rc1.dev110+gacb0f1dcd-tp4-pp4-93377ccd", "usage": {"prompt_tokens": 9, "total_tokens": 43, "completion_tokens": 34, "prompt_tokens_details": {"cached_tokens": 0, "created_cache_tokens": 0, "multimodal_tokens": null}}, "prompt_logprobs": null, "prompt_token_ids": null, "prompt_text": null, "kv_transfer_params": null, "ec_transfer_params": null, "metrics": null}]
```



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Thank you for your attention.

Questions?

START HERE
↓

